USO · TUTORIAT 01 DIN 10

Primii pași în Linux

Ce face un sistem de operare, unde stau fișierele și cum vorbim cu calculatorul direct din terminal.

```
student@fmi:~$ echo "Salut, USO!"
```

PLANUL DE AZI

Cinci întrebări, apoi trecem la terminal

1

Din ce e făcut un sistem?

Straturile: de la aplicații până la hardware.

2

Cine are voie la hardware?

User mode, kernel mode și „ușa” dintre ele.

3

Unde stau fișierele?

Un singur arbore, căi absolute și relative.

4

Cum găsește shell-ul o comandă?

Variabila `$PATH` și de ce scriem `./`.

5

Cum devine codul C program?

Pașii lui `gcc` și codul de ieșire.

Cum împărțim cele 90 de minute



1 · STRATURILE

Un calculator e construit pe etaje

Fiecare etaj vorbește doar cu vecinul lui. Apasă pe un etaj.

Aplicații — browser, editor, programul tău C

Shell și utilitare — bash, ls, cp, gcc

APELURI DE SISTEM

Kernel (nucleul) — procese, memorie, fișiere

Hardware — procesor, RAM, disc, rețea

INIMA SISTEMULUI

Kernelul

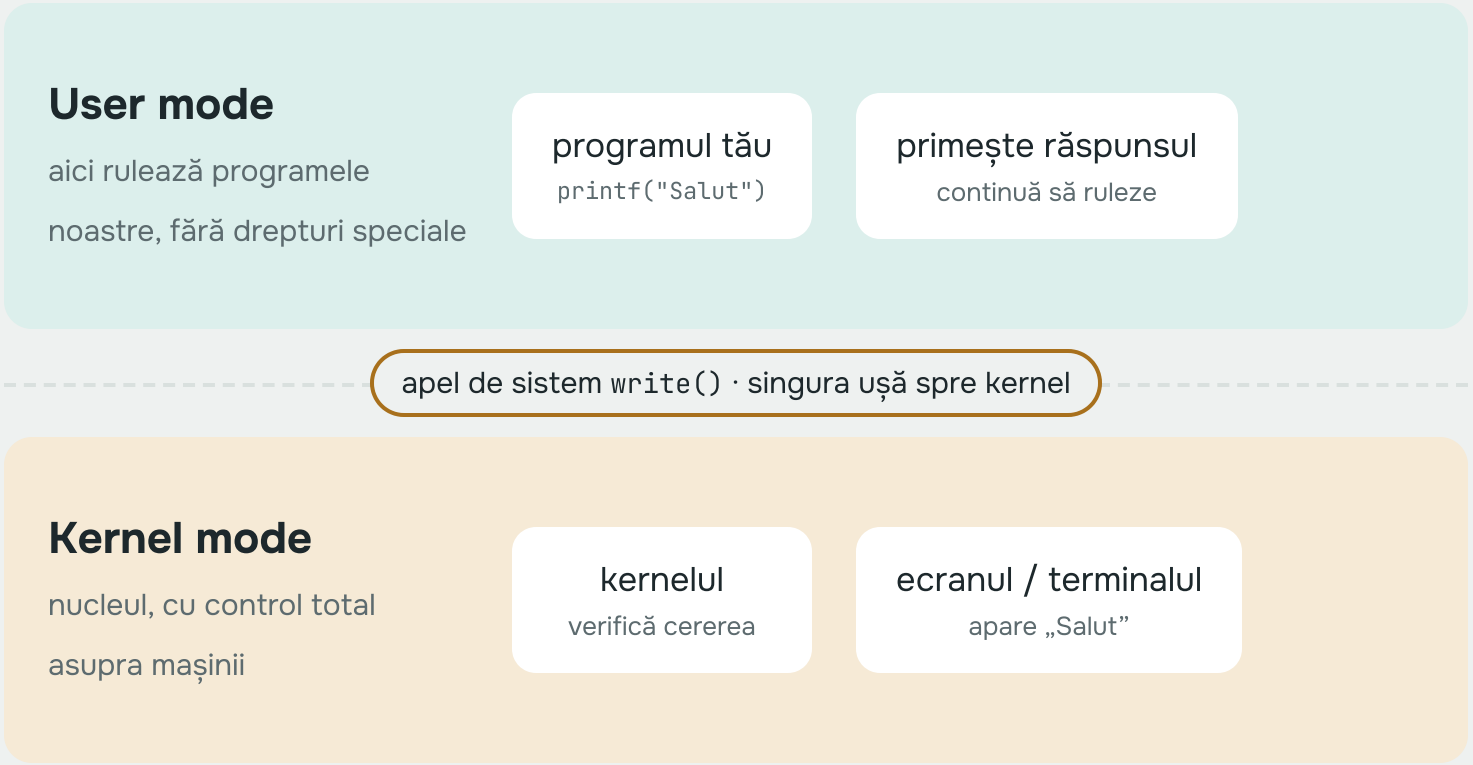
Nucleul sistemului de operare. Împarte procesorul între programe, ține evidența memoriei și a fișierelor, vorbește cu dispozitivele prin drivere.



Ca la restaurant: bucătarul-șef: doar el folosește aragazul și cuțitele.

2 · CINE ARE VOIE LA HARDWARE

Programul tău nu atinge direct ecranul. Cere voie.



1. Programul vrea să afișeze ceva, dar nu are voie să scrie singur pe ecran.
2. Face un **apel de sistem**. Procesorul trece în kernel mode.
3. Kernelul verifică cererea și trimite textul către terminal.
4. Procesorul revine în user mode, iar programul merge mai departe.

Pas următor | De la capăt

Dacă greșește un program
Se oprește doar el (de exemplu `Segmentation fault`). Restul sistemului merge.

Dacă greșește kernelul
Cade tot calculatorul: *kernel panic*. De asta kernelul e protejat.

3 · UNDE STAU FIȘIERELE

Totul pornește dintr-un singur punct: /

Nu există `C:\` sau `D:\`. Un stick USB apare și el ca un folder, de exemplu `/media/student/USB`.



/home · casele utilizatorilor

Fiecare utilizator are aici folderul lui. Al tău este /home/student, pe scurt ~. Aici lucrezi aproape tot timpul.

Ce găsești înăuntru

- `student/`
- `student/Desktop`
- `student/tutoriat-01`

3 · CĂI ABSOLUTE ȘI RELATIVE

Două feluri de a da indicații

Absolută: adresa completă, de la /. Relativă: „de unde ești acum, mergi…”. Alege un fișier sau folder din arbore.

- /
  - etc
    - passwd
  - tmp
  - home
    - student
      - documente
      - proiect
        - Makefile
        - src — ești aici
          - main.c

EȘTI ÎN	/home/student/proiect/src
ȚINTA	Makefile
ABSOLUTĂ	/home/student/proiect/Makefile
RELATIVĂ	../Makefile

`cd ../Makefile` Ținta e un fișier. Poți intra doar în foldere.

. folderul curent	.. un nivel mai sus	~ folderul tău acasă	/ rădăcina

BONUS · MĂȘTI DE CĂUTARE

Cum alegi mai multe fișiere deodată

`*` = orice, oricât de lung · `?` = exact un caracter · `[12]` = unul dintre cele din paranteze.

$ ls *.c

*.c | tema?.txt | lab[12].c | *a* | *.o

main.c | util.c | util.h | lab1.c | lab2.c | lab3.c | tema1.txt | tema2.txt | tema10.txt | notite.md | Makefile | date.in

Ce rulează de fapt, după ce shell-ul înlocuiește tiparul:

```
# tu scrii:
$ ls *.c
# shell-ul pornește de fapt:
$ ls main.c util.c lab1.c lab2.c lab3.c
```

> Înlocuirea o face **shell-ul**, înainte să pornească comanda. `ls` (sau `rm`, sau orice altă comandă) nu vede niciodată steluța, ci direct lista de fișiere.

TERMINALUL

Fereastra în care scrii comenzi

Consola text · /dev/tty1

Ecran negru, fără ferestre

O primești direct de la kernel, chiar dacă nu pornește interfața grafică. O deschizi cu `Ctrl+Alt+F1…F6`.

Pseudoterminal · /dev/pts/0

Terminalul dintr-o fereastră

Aplicația „Terminal”, un tab din `tmux` sau o conexiune SSH. Aproape mereu vei lucra în așa ceva.

Apasă pe o comandă ca s-o „rulezi”

- `whoami` · cine sunt?
- `tty` · în ce terminal?
- `pwd` · unde sunt?
- `echo $HOME` · unde e „acasă”?

```
student@fmi:~/tutoriat-01$ whoami
student
student@fmi:~/tutoriat-01$
```

4 · CUM GĂSEȘTE SHELL-UL O COMANDĂ

De ce scriem `./program` și nu doar `program`?

Când scrii doar un nume, shell-ul îl caută pe rând în folderele din `$PATH`. Folderul tău nu e pe listă.

`ls` `gcc` `program` `./program`

`$PATH =` `/usr/local/bin` : `/usr/bin` : `/bin` : `/usr/games`

Alege o comandă de mai sus și urmărește căutarea.

> **`./` înseamnă „uite exact fișierul, aici”.** Când comanda conține un `/`, shell-ul nu mai caută în `$PATH`.

> **De ce nu e `.` în `$PATH`?** Cineva ar putea lăsa în `/tmp` un program fals numit `ls`. Ai rula programul lui fără să-ți dai seama.

5 · DE LA COD LA PROGRAM

`gcc` face patru pași, unul după altul

De obicei îi faci pe toți dintr-o comandă. Apasă pe fiecare etapă ca să vezi cum arată fișierul.

`prog.c` codul tău → **`prog.i`** preprocesare → **`prog.s`** compilare → **`prog.o`** asamblare → **`program`** legare

```c
#include <stdio.h>

// primul program
int main(void) {
    printf("Salut!\n");
    return 0;
}
```

Codul sursă

Ce ai scris tu: text simplu, pe care îl citește un om.

Ca să te oprești după pasul ăsta:

```
$ nano prog.c
```

5 · COMPILĂM ȘI RULĂM EXERCIȚIUL

Fiecare program lasă în urmă un număr

`0` înseamnă „a mers”. Orice altceva înseamnă „ceva n-a mers”. Îl vezi cu `echo $?`.

Comanda pe care o folosim la laborator. Apasă pe o opțiune:

`-Wall -Wextra` `-std=c11` `-pedantic` `-g` `-o exercitiu`

-Wall -Wextra · Compilatorul te avertizează despre lucruri suspecte: variabile nefolosite, comparații ciudate. Citește avertismentele, de obicei au dreptate.

Rulează exercițiul în trei feluri:

`./exercitiu date.in`

`./exercitiu`

`./exercitiu lipsa.txt`

```
$ gcc -Wall -Wextra -std=c11 -pedantic -g -o exercitiu exercitiu.c
$ ./exercitiu date.in
[+] Director personal   : /home/student
...
Analizam fisierul: date.in
[ 1] (11 cuv,  64 B): Linux este un nucleu de sistem de operare...
...
  [+] Linii citite     : 8
  [+] Cuvinte gasite   : 98

[SUCCES] Programul s-a incheiat cu succes.
$ echo $?
0   # totul a mers
```

FIȘĂ RAPIDĂ

Comenzile pe care le folosești azi

Apasă pe o comandă ca să vezi un exemplu.

pwd unde sunt	**ls -la** ce e aici	**cd** schimbă folderul	**mkdir** folder nou	**touch** fișier gol	**cat** arată conținutul
cp copiază	**mv** mută / redenumește	**rm** șterge	**rmdir** șterge folder gol	**echo** afișează text	**which** unde e programul

```
$ ls -la
drwxr-xr-x  schelet
drwxr-xr-x  solutii
-rw-r--r--  README.md
```

ls -la

Listează tot din folder, inclusiv fișierele ascunse (cele care încep cu punct), cu detalii.

GREȘELI FRECVENTE

Mesajele pe care le vei vedea sigur

Apasă pe o eroare: vezi de ce apare și cum o rezolvi.

`bash: ./program: Permission denied` ⌄

DE CE

Fișierul nu are dreptul de execuție (x). Linux nu rulează orice fișier, doar pe cele marcate ca executabile.

REZOLVARE

`chmod +x program`

`bash: program: command not found` ›

`rmdir: failed to remove 'teme': Directory not empty` ›

`cat: Date.in: No such file or directory` ›

`cd: tutoriat-01: No such file or directory` ›

ÎNCĂLZIRE ÎNAINTE DE KAHOOT

Trei întrebări rapide

Ești în /home/student/proiect. Unde te duce cd ..?

- /home/student
- /home
- /

Ce afișează echo $? după un program care a mers fără probleme?

- 1
- 0
- OK

Cine transformă *.c în lista de fișiere?

- comanda rm
- kernelul
- shell-ul

ACUM E RÂNDUL TĂU

Deschide terminalul și urmează pașii

Completează în `exercitiu.c` zonele marcate cu `TODO`. Bifează pe măsură ce avansezi.

- [ ] `cd tutoriat-01/schelet` intră în folder
- [ ] `ls -la` vezi ce fișiere ai
- [ ] `nano exercitiu.c` rezolvă TODO-urile
- [ ] `make` compilează
- [ ] `./exercitiu date.in` rulează
- [ ] `echo $?` a mers? ar trebui 0

Ce exersezi

Citești argumentele programului (`argc`, `argv`), variabile de mediu cu `getenv()`, un fișier linie cu linie și întorci un cod de ieșire corect.

Te-ai blocat? Soluția completă e în `tutoriat-01/solutii/`. Încearcă întâi singur.